EXPERIMENT 15

CareerPathwaysandtheFuture of Low-Code Development

Industry-WideStudy:Report andPresentation Outline

Sreeman Naibal S
241401106

1. Aim

To study the career paths open to low-code developers and the direction low-code development is likely to take across the industry, looking at low-code and no-code platforms in general rather than one product. The study ends with a report and a presentation outline that link these trends to the skills practised in the course experiments.

The study helps in deciding which skills to build next and which roles to aim for, as low-code, pro-code and AI-assisted development move closer together.

2. Procedure

Step 1: Trace the history. How application building moved from hand coding to low-code and then to AI-assisted building was drawn as a timeline (Section 3.1).

Step 2: Survey the platforms. Well-known low-code platforms were compared by focus and typical use, to show that the field is wider than any single product (Section 3.2).

Step 3: Identify the trends. Current industry trends were listed with their effect on careers (Section 3.3), and a likely future workflow was drawn (Section 3.9).

Step 4: Map the careers. Roles were arranged from entry to senior level, with possible switches to other careers, as a pathway diagram and a role table (Sections 3.4 and 3.5).

Step 5: Build the skill stack and roadmap. The skills needed were arranged in layers and turned into a step-by-step learning plan (Sections 3.6 and 3.7).

Step 6: Examine the risks. Shadow IT, security, lock-in, AI output quality and a changing job market were listed with ways to handle each (Section 3.8).

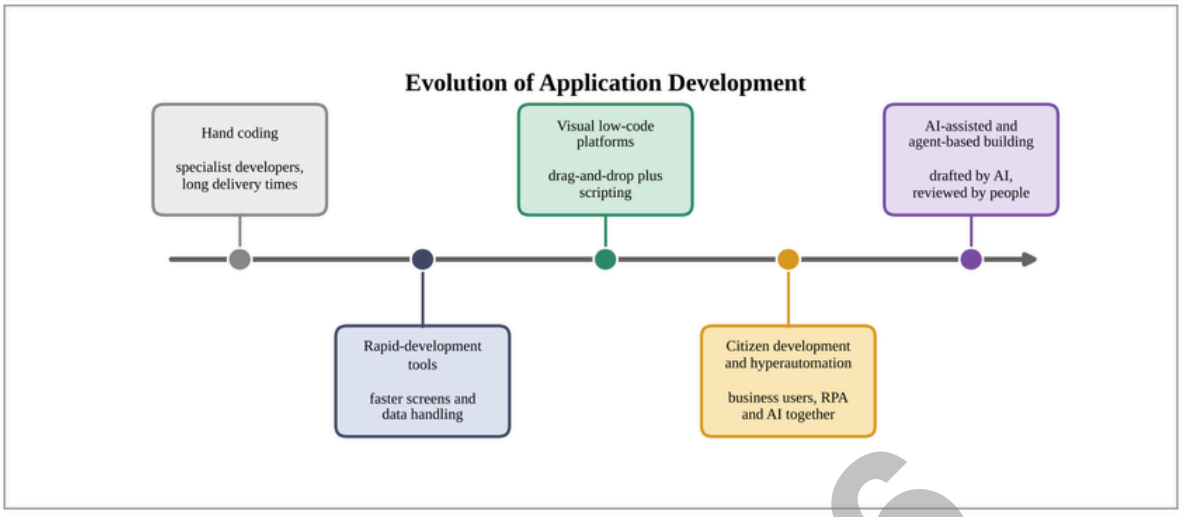
Step7:Preparethepresentation. A slide-by-slide outline was drawn up so that the findings can be presented clearly (Section 3.10).

Note: this study is based on general knowledge of the industry. Trends and any forecast figures change quickly and should be checked against current, original sources before they are quoted as facts. The timeline, roles and roadmap are a general guide, not a prediction for any one company.

241401106

3. Output

3.1 Evolution of application development



The timeline moves from hand coding to rapid-development tools, then to visual low-code platforms, then to citizen development with hyperautomation, and now to AI-assisted and agent-based building. Effort stepped up the effort needed to produce an application, but also increased the need for review and oversight.

3.2 The platform landscape beyond Zoho Creator

Low-code is a wide field. The platforms below are examples of different focus areas, and many skills carry over from one to another.

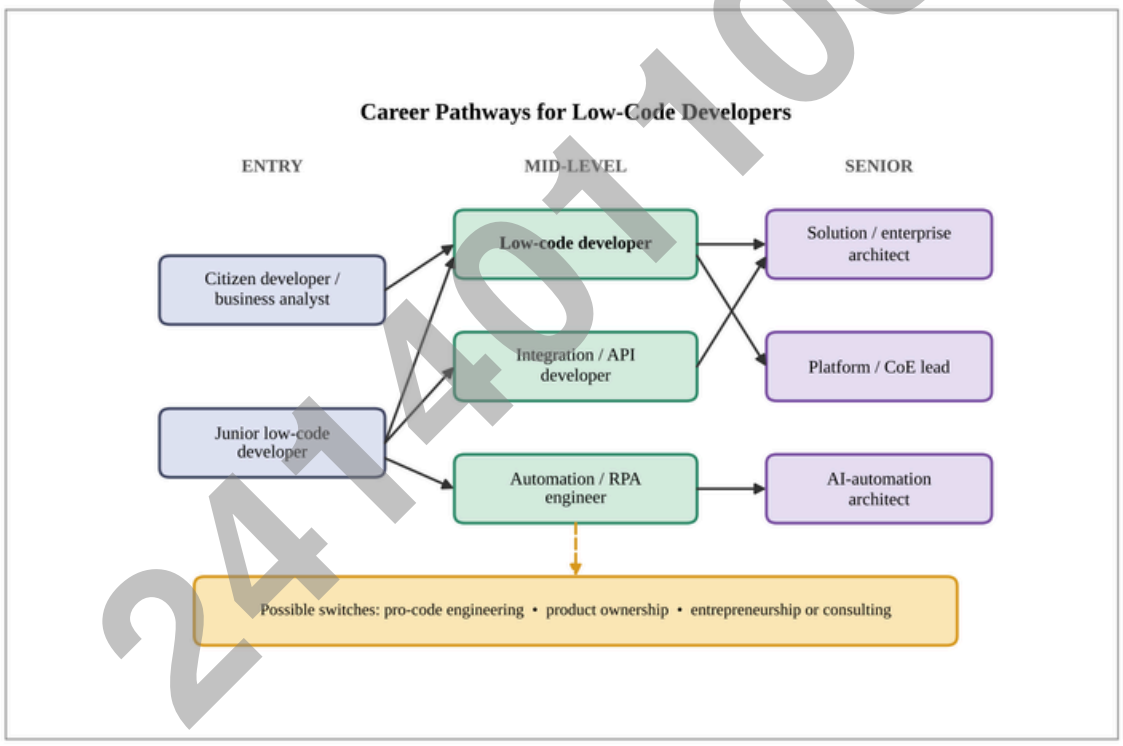
Platform	*General focus*	*Typical use*
Zoho Creator	Business apps closely linked to the Zoho product family.	Process apps for small and mid-size businesses.
Microsoft Power Apps	Apps tied to Microsoft 365 and Azure services.	Internal tools in organisations that already use Microsoft.
OutSystems	High-scale development with strong support for custom code.	Large, complex enterprise applications.
Mendix	Model-driven enterprise application platform.	Enterprise and industrial applications.
Appian	Process automation and case management.	Workflow-heavy, regulated processes.
Salesforce Platform	Custom apps built around customer data.	Customer-facing and sales-related applications.
Google AppSheet	Apps built from spreadsheets and other data sources.	Simple team tools and field data collection.

241401106

3.3 Trends shaping the future

Trend	What it means	Effect on careers
AI-assisted building	An app or a piece of logic can be drafted from a plain-language description.	Less routine building, more reviewing, design and testing.
AI agents inside apps	Agents take actions within workflows instead of only giving suggestions.	New work in designing, limiting and monitoring agents.
Governed citizen development	Business users build apps within rules set by IT.	Demand for platform owners and centres of excellence.
Hyperautomation	Low-code, RPA and AI are joined to automate whole processes.	Automation engineers and process designers gain value.
Legacy modernisation	Old systems are wrapped or replaced with low-code front ends.	Integration and migration skills become important.
API-first, composable apps	Apps are assembled from services reached through APIs.	API design and integration skills transfer across platforms.
Fusion teams	Business and technical staff build solutions together.	Communication and requirement gathering matter as much as tools.

3.4 Career pathways



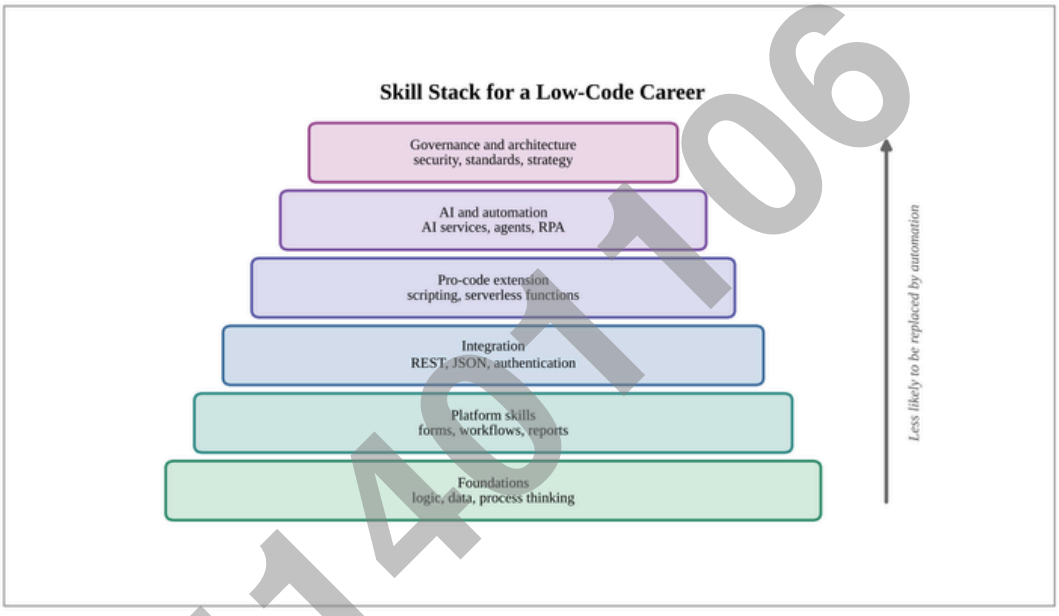
The diagram shows common routes from entry roles such as citizen developer or junior low-code developer, through mid-level roles in development, integration and automation, to senior roles such as architect, platform lead or AI-automation architect. The dashed arrow marks possible switches into pro-code engineering, product ownership or entrepreneurship.

241401106

3.5 Roles at a glance

Role	What the person does	Key skills
Citizen developer / analyst	Builds simple apps for their own team and gathers requirements.	Process thinking, basic platform use.
Low-code developer	Builds forms, workflows, reports and scripts.	Platform skills, data modelling, scripting.
Integration / API developer	Connects apps to other systems through APIs.	REST, JSON, authentication, error handling.
Automation / RPA engineer	Automates whole processes, including AI steps.	Workflow design, RPA tools, AI services.
Solution / enterprise architect	Designs the overall solution, its security and governance.	Architecture, security, platform strategy.
Platform / CoE lead	Sets standards and supports citizen developers.	Governance, training, communication.

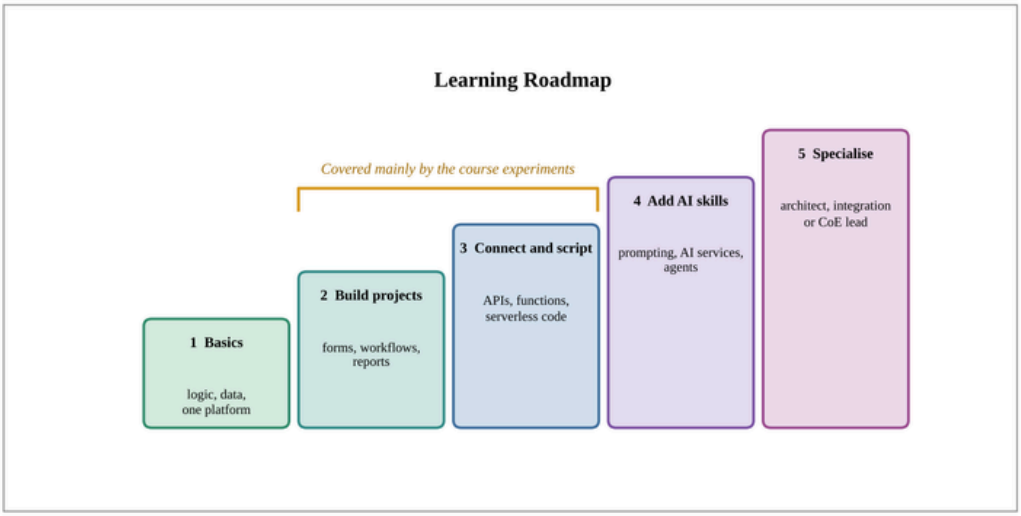
3.6 Skill stack for a low-code career



Skills are arranged in layers, from foundations such as logic and data, through platform and integration skills and pro-code extension, up to AI, automation and governance. Each layer adds value on top of the one below it, and the upper layers are the ones least likely to be replaced by automation.

241401106

3.7 Learning roadmap

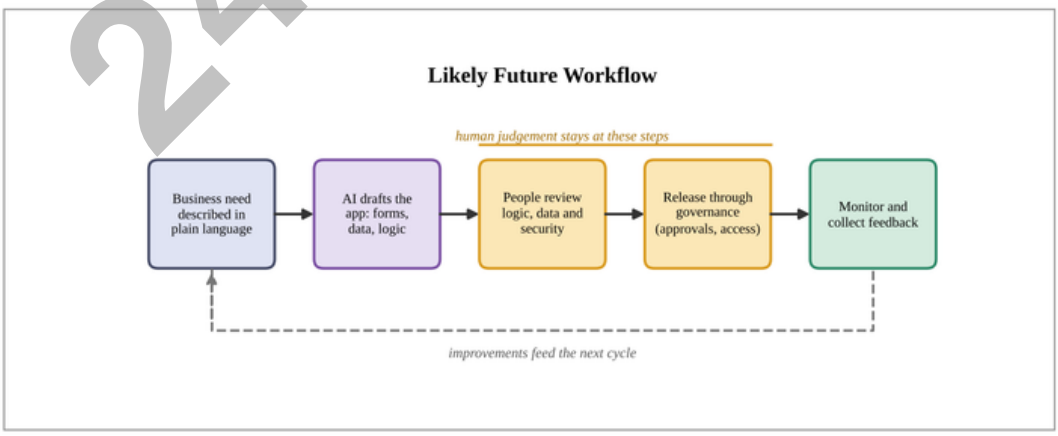


The roadmap goes from basics to projects, then to connecting and scripting, then AI skills, and finally specialisation. The course experiments on building an appointment-style application, workflows and pro-code extension fall mainly in steps two and three.

3.8 Risks and challenges

Challenge	*Why it matters*	*How to handle it*
Shadow IT	Apps built outside IT may be unknown, unsecured or duplicated.	Central app register, approval steps, a centre of excellence.
Security and compliance	Easy building can hide weak access control and data exposure.	Role-based access, reviews, testing and audit trails.
Quality of AI-generated output	Generated logic can look correct while hiding errors.	Human review, test cases and version control.
Vendor lock-in	Apps depend on one platform's tools and pricing.	Keep logic in APIs, keep data exportable, learn transferable skills.
Changing job market	Routine build tasks may shrink as AI improves.	Move toward design, integration, governance and AI oversight.

3.9 Likely future workflow



In the likely future workflow a business need is described in plain language and AI drafts the application. People then review the logic, data and security, the application is released through governance, and monitoring feeds the next round of improvements. Human judgement stays at the review and release steps.

241401106

3.10 Presentation outline

The findings can be presented in ten slides, with one idea per slide.

Slide	*Title*	*Key points*
1	Title and introduction	Topic, name and register number.
2	What is low-code?	Visual building with the option to add code; link to the course application work.
3	How development evolved	Timeline from hand coding to AI-assisted building.
4	Platform landscape	Zoho Creator, Power Apps, OutSystems, Mendix, Appian, Salesforce, AppSheet.
5	Future trends	AI-assisted building, agents, governed citizen development, hyperautomation.
6	Career pathways	Pathway diagram from entry to senior roles and possible switches.
7	Skills and roadmap	Skill stack and the five-step learning roadmap.
8	Risks and challenges	Shadow IT, security, AI output quality, lock-in, job market.
9	My practical work	The course experiments as hands-on examples of low-code and pro-code work.
1	Conclusion and questions	Low-code and pro-code will work together; keep learning.

4. Final Result

The study showed that low-code is growing from a quick way to build simple apps into a wider field where visual building, pro-code extension and AI work side by side. The platform landscape is broad, and skills learned on one platform, such as Zoho Creator, carry over to others.

Career paths run from citizen developer and junior low-code developer, through integration and automation roles, to architect, platform lead and AI-automation roles, with routes into pro-code engineering and product work. The skill stack and roadmap show where the course experiments fit on this path.

The main risks are shadow IT, security gaps, the quality of AI-generated output and lock-in, and they are managed through governance and human review. The exercise met its objective, and the presentation outline is ready to be turned into slides. The conclusions rest on general industry knowledge, so any trend or statistic should be checked against current original sources before use.

241401106